UNIVERSIDAD AUT?NOMA DE NUEVO LE?N

Facultad de Ciencias F?sico Matem?ticas
Licenciatura en Ciencias Computacionales

DESARROLLO WEB BACK-END

Unidad 08

Transacciones, concurrencia y consultas controladas

Reto: admisi?n concurrente con cupo ?nico

Alumno
Leonardo Elizondo N??ez

Monterrey, Nuevo Le?n a 11 de noviembre de 2026

OBJETIVO

A?adir materias e inscripciones al sistema persistente y proteger la regla de cupo ante admisiones concurrentes. El reto prepara una materia de un solo lugar y dos alumnos distintos, lanza dos tareas Java contra el bean real del servicio y comprueba que solo una inscripci?n se confirme.

CAMBIOS DE LA UNIDAD

Archivo / componente	Modificaci?n y finalidad
src/main/resources/db/migration/V2__crear_materias_e_inscripciones.sql	Crea materias e inscripciones, claves for?neas, par ?nico alumno/materia e ?ndice por materia. Inserta MAT101 con cupo de laboratorio.
repository/jpa/MateriaEntity.java y MateriaRepository.java	Mapean la oferta acad?mica y exponen la consulta de materia con bloqueo pesimista de escritura.
repository/jpa/InscripcionEntity.java y InscripcionRepository.java	Modelan las relaciones alumno/materia, fecha, conteo por materia, duplicados y consultas paginadas.
service/InscripcionService.java	Inscribir es una transacci?n READ_COMMITTED: bloquea materia, valida alumno activo/duplicado/cupo y guarda; los fallos revierten.
controller/InscripcionController.java	Expone alta e historial; valida ids, tama?o m?ximo de p?gina y lista blanca de orden con desempate estable por id.
dto/InscripcionResponse.java y PaginaResponse.java	Devuelven un historial paginado sin exponer entidades JPA.
exception/OperacionAcademicaException.java y GlobalExceptionHandler.java	Traducen falta de cupo, duplicado, alumno inactivo, materia ausente y referencias protegidas a respuestas coherentes.
requests.http	Incluye ejemplos HTTP para inscribir y consultar historial paginado.
src/test/.../InscripcionConcurrentePostgresTest.java	En reto-08, crea datos ?nicos, coordina dos tareas y prueba el bloqueo real del motor PostgreSQL.

EJECUCI?N CONCURRENTE Y LUGAR DE ESPERA

El servicio usa @Transactional(isolation = READ_COMMITTED) y llama a MateriaRepository.bloquearPorId, anotado con PESSIMISTIC_WRITE. Hibernate emite un bloqueo de fila equivalente a SELECT ... FOR UPDATE. Las tareas esperan en PostgreSQL por la misma fila de materia; despu?s del commit de la primera, la segunda vuelve a evaluar el cupo y recibe SIN_CUPO. No hay un monitor synchronized ni un candado local en el servicio, de modo que la coordinaci?n pertenece a la base y funciona tambi?n con varias instancias de la aplicaci?n.

La prueba exige DB_URL con esquema jdbc:postgresql, crea una materia con cupo 1 y dos alumnos ?nicos, y libera dos tareas mediante CountDownLatch. Mantiene temporalmente un bloqueo de fila para que ambas conexiones queden esperando y consulta pg_stat_activity hasta observar dos sesiones con wait_event_type=Lock y una consulta a materias. Tras liberar la fila, valida una admisi?n, un rechazo SIN_CUPO y una sola fila persistida.

```
SELECT count(*)
FROM pg_stat_activity
WHERE wait_event_type = 'Lock'
  AND lower(query) LIKE '%materias%';
```

VALIDACI?N EJECUTADA

mvnw.cmd test: 9 pruebas descubiertas, 8 exitosas y 1 omitida. La prueba concurrente se omite si DB_URL no apunta a PostgreSQL. En este equipo no est?n instalados Docker ni psql, as? que no se pudo ejecutar la prueba contra PostgreSQL; no se presenta el resultado de concurrencia como observado. En un laboratorio PostgreSQL, establecer DB_URL, DB_USER y DB_PASSWORD y ejecutar mvnw.cmd -Dtest=InscripcionConcurrentePostgresTest test.

REPOSITORIO Y FECHA

Repositorio: Leoxd23/Cuaderno_Back-End. La base de Unidad 8 se registr? en master; la prueba del reto est? en la rama reto-08. Fecha solicitada para este informe: 11 de noviembre de 2026.